MAHATMA EDUCATION SOCIETY'S

PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE
(Autonomous)

NEW PANVEL

PROJECT REPORT ON

"POEM GENERATOR BASED ON THEME USING NLP"

IN PARTIAL FULFILLMENT OF

MASTER OF **DATA ANALYTICS PART-II**

SEMESTER III – 2025-26

PROJECT GUIDE

PROF. SHIVAPRADEEP MUTHUPANDI

SUBMITTED BY: **SHRUTI N YELVE**

ROLL NO: **4845**

Mahatma Education Society's

PILLAI COLLEGE OF ARTS,COMMERCE & SCIENCE

(Autonomous)

Re-accredited "A" Grade by NAAC (3rd Cycle)

Project Completion Certificate

THIS IS TO CERTIFY THAT

SHRUTI YELVE

of **M.Sc. Data Analytics Part - II** has completed the project titled **POEM GENERATOR BASED ON THEME USING NLP** of subject **Natural language processing** under our guidance and supervision during the academic year 2025-26 in the department of

PROJECT GUIDE **COURSE CORDINATOR** **HEAD OF DEPARTMENT**

Pillai College of Arts, Commerce & Science (Autonomous) New Panvel

INTRODUCTION

The rapid advancement of Artificial Intelligence (AI) and Natural Language Processing (NLP) has transformed the way humans create, understand, and interact with text. Among the many creative applications of NLP, automated poetry generation has emerged as a fascinating field that blends computational linguistics, deep learning, and artistic expression. A *Poem Generator based on Theme using NLP* is an intelligent system designed to generate poetic compositions automatically based on a given theme, emotion, or prompt supplied by the user. This technology represents the intersection of computational creativity and human-like text generation, enabling machines to emulate literary styles, produce expressive language, and even mimic poetic devices such as metaphors, rhyme, rhythm, and imagery.

Traditionally, poetry has been considered one of the most nuanced, expressive, and complex forms of human literature. Crafting a poem requires not just linguistic knowledge but also emotional depth, creativity, and cultural context. For decades, researchers questioned whether machines could ever replicate such human-like artistry. However, breakthroughs in NLP—particularly with models like RNNs, LSTMs, Transformers, and large-scale pre-trained language models—have proven that poetic text generation is not only possible but also increasingly sophisticated. These models learn patterns from vast amounts of text data and can generate coherent, meaningful, and stylistically rich verses based on user-defined themes such as “love,” “nature,” “loneliness,” “hope,” or “friendship.”

Some advanced systems even allow customization of poetic structure, such as haiku, sonnet, free verse, or rhyming couplets.

CODE AND OUTPUT

Poem_generator.py

```python
import re
import random
import nltk
from nltk.corpus import wordnet as wn

# ----------------------------------------------------
# NLTK downloads (safe to run; no-op if already present)
# ----------------------------------------------------
def ensure_nltk():
    try:
        wn.synsets("love")
    except LookupError:
        nltk.download("wordnet", quiet=True)
        nltk.download("omw-1.4", quiet=True)
    # Optional: cmudict (used by the app for meter-aware shaping; falls ba
    try:
        from nltk.corpus import cmudict  # noqa: F401
        _ = nltk.corpus.cmudict.dict()
    except LookupError:
        try:
            nltk.download("cmudict", quiet=True)
        except Exception:
            pass
```

```python
ensure_nltk()

# ----------------------------------------------------
# Helpers
# ----------------------------------------------------
def clean(word: str) -> str:
    w = (word or "").lower().strip()
    return re.sub(r"[^a-zA-Z]", "", w)

def clean_theme(theme: str) -> str:
    theme = re.sub(r"[^a-zA-Z\s]", " ", theme or "")
    theme = re.sub(r"\s+", " ", theme).strip().lower()
    return theme.split()[0] if theme else ""

def capitalize(s: str) -> str:
    return (s[0].upper() + s[1:]) if s else s

# ----------------------------------------------------
# Theme typing with WordNet
# ----------------------------------------------------
def detect_theme_type(theme: str) -> str:
    syns = wn.synsets(theme)
    if not syns:
```

```python
VOCAB = {
    "nature": {
        "nouns": ["forest", "river", "wind", "sky", "valley", "mountain", "leaf",
                  "horizon", "stone", "sunlight", "meadow", "shore"],
        "adjs":  ["quiet", "endless", "golden", "wild", "gentle", "ancient",
                  "soft", "blooming", "shimmering"],
        "verbs": ["whispers", "drifts", "glows", "stirs", "breathes", "moves", "dances", "war
    },
    "emotion": {
        "nouns": ["heart", "memory", "feeling", "silence", "dream", "moment", "soul", "though
        "adjs":  ["tender", "quiet", "warm", "soft", "aching", "gentle", "deep", "delicate"],
        "verbs": ["holds", "follows", "reminds", "echoes", "lingers", "awakens", "touches"],
    },
    "relationship": {
        "nouns": ["friend", "hand", "laughter", "voice", "shoulder", "path", "bond", "comfort
        "adjs":  ["true", "steady", "kind", "faithful", "close", "warm"],
        "verbs": ["walks", "stays", "guides", "listens", "stands", "joins", "reaches"],
    },
    "object": {
        "nouns": ["stone", "doorway", "street", "window", "shadow", "signal", "bridge", "lant
        "adjs":  ["faded", "quiet", "broken", "clear", "simple", "dusty"],
        "verbs": ["waits", "settles", "rests", "holds", "marks"],
    },
```

```python
def rhyme_key(word: str) -> str:
    w = clean(word)
    return w[-2:] if len(w) > 2 else w

def select_rhymes(pool, scheme="AABB", lines=6):
    ends = []
    rhymes = {}
    for w in pool:
        rhymes.setdefault(rhyme_key(w), []).append(w)
    keys = [k for k, v in rhymes.items() if len(v) >= 2]
    if not keys:
        return random.choices(pool, k=lines)
    k1 = random.choice(keys); k2 = random.choice(keys)
    if scheme == "AABB":
        A = random.choice(rhymes[k1]); B = random.choice(rhymes[k2])
        ends = [A, A, B, B]
    else:  # ABAB
        A = random.choice(rhymes[k1]); B = random.choice(rhymes[k2])
        ends = [A, B, A, B]
    while len(ends) < lines:
        ends.append(random.choice(pool))
    return ends[:lines]
```

```python
def generate_poem(theme, lines=6, emotion="calm", rhyme_scheme="AABB"):
    theme = clean_theme(theme)
    theme_type = detect_theme_type(theme or "truth")

    nouns = VOCAB[theme_type]["nouns"]
    adjs  = VOCAB[theme_type]["adjs"] + EMOTION_TONES.get(emotion, [])
    verbs = VOCAB[theme_type]["verbs"]

    rhyme_pool = nouns + adjs
    rhyme_ends = select_rhymes(rhyme_pool, rhyme_scheme, lines)

    poem_lines = []
    chosen = random.sample(TEMPLATES, k=min(lines, len(TEMPLATES)))
    while len(chosen) < lines:
        chosen.append(random.choice(TEMPLATES))

    for i, temp in enumerate(chosen[:lines]):
        line = temp.format(
            theme=theme or "truth",
            noun=random.choice(nouns),
            noun2=random.choice(nouns),
            adj=random.choice(adjs),
```

```python
            noun=random.choice(nouns),
            noun2=random.choice(nouns),
            adj=random.choice(adjs),
            adj2=random.choice(adjs),
            verb=random.choice(verbs),
            verb2=random.choice(verbs),
            simile=random.choice(SIMILES),
            metaphor=random.choice(METAPHORS),
        )
        parts = line.rstrip(".").split()
        if parts:
            parts[-1] = rhyme_ends[i]
            line = " ".join(parts)
        poem_lines.append(capitalize(line) + ".")

    title = capitalize(theme or "Poem")
    return f"{title}\n" + "\n".join(poem_lines)
```

App.py

```python
# app.py
import io
import re
import textwrap as tw
from pathlib import Path

import streamlit as st
from poem_generator import generate_poem  # your NLP engine (WordNet + curated vocab +

# Optional CMUdict for syllables (fallback to heuristic if missing)
try:
    from nltk.corpus import cmudict
    _CMU = cmudict.dict()
except Exception:
    _CMU = None

st.set_page_config(page_title="🪄 Poem Studio Ultra", page_icon="🪄", layout="wide")

# ---------------------------
# Accent color selector
# ---------------------------
ACCENTS = {
    "Aqua":   ("#00e5ff", "#80ffd3"),
    "Purple": ("#9b5cff", "#ff73fa"),
```

```
DARK = """
<style>
:root { --bg1:#0b0f17; --bg2:#151b2e; --text:#eaf6ff; --muted:#a9bed0;
        --card: rgba(255,255,255,0.06); --border: rgba(255,255,255,0.18);
        /* --a1 and --a2 are injected separately */ }
html, body, [class*="css"] {
  background: radial-gradient(1200px 600px at 20% 10%, rgba(0,229,255,.06), transparent 60%),
              radial-gradient(800px 400px at 80% 20%, rgba(128,255,211,.05), transparent 50%)
              linear-gradient(-45deg,var(--bg1),var(--bg2),#0e2535,#121a2a);
  background-size: 400% 400%;
  animation: shift 22s ease infinite;
  color: var(--text) !important;
}
@keyframes shift { 0%{background-position:0% 50%} 50%{background-position:100% 50%} 100%{bac
.block-container { padding-top: 1.6rem; padding-bottom: 2rem; }
.hero { font-weight:800; font-size:2.1rem; letter-spacing:.4px;
        background:linear-gradient(90deg, var(--a1), var(--a2));
        -webkit-background-clip:text; background-clip:text; color:transparent; }
.card { background: var(--card); border:1px solid var(--border); border-radius:18px; padding
        box-shadow:0 24px 50px -20px rgba(0,0,0,.55),0 0 24px rgba(0,229,255,.12); }
.badge { display:inline-block; padding:6px 10px; margin-top:4px; border-radius:999px;
         border:1px solid rgba(0,229,255,.30); box-shadow:0 0 10px rgba(0,229,255,.15);
         color:var(--muted); font-size:.85rem; }
```

```
LIGHT = """
<style>
:root { --bg1:#f8fbff; --bg2:#eaf5ff; --text:#0f2030; --muted:#456;
        --card: rgba(255,255,255,0.9); --border: rgba(0,0,0,0.08);
        /* --a1 and --a2 are injected separately */ }
html, body, [class*="css"] {
  background: radial-gradient(1000px 500px at 15% 15%, rgba(0,97,255,.05), transparent 55%
              radial-gradient(700px 380px at 82% 18%, rgba(41,171,226,.06), transparent 50
              linear-gradient(-45deg,var(--bg1),var(--bg2),#f2f8ff,#eef6ff);
  background-size: 400% 400%;
  animation: shift 22s ease infinite;
  color: var(--text) !important;
}
@keyframes shift { 0%{background-position:0% 50%} 50%{background-position:100% 50%} 100%{b
.block-container { padding-top: 1.6rem; padding-bottom: 2rem; }
.hero { font-weight:800; font-size:2.1rem; letter-spacing:.4px;
        background:linear-gradient(90deg, var(--a1), var(--a2));
        -webkit-background-clip:text; background-clip:text; color:transparent; }
.card { background: var(--card); border:1px solid var(--border); border-radius:18px; paddi
        box-shadow:0 18px 36px -18px rgba(0,0,0,.18); }
.badge { display:inline-block; padding:6px 10px; margin-top:4px; border-radius:999px; bord
         color:var(--muted); font-size:.85rem; }
.button-primary button { background:linear-gradient(90deg,var(--a1),var(--a2))!important;
                         font-weight:700!important; border-radius:12px!important; padding:
```

```python
with st.sidebar:
    st.subheader("🎨 Style")
    accent_name = st.selectbox("Accent", list(ACCENTS.keys()), index=0)
    a1, a2 = ACCENTS[accent_name]
    st.write(":small_blue_diamond: Accent:", accent_name)

if "light_mode" not in st.session_state:
    st.session_state["light_mode"] = False
st.session_state["light_mode"] = st.toggle("🌓 Light / Dark theme", value=st.session_state["

# Inject accent variables first, then theme CSS (prevents string-escape issues)
accent_css = f"""
<style>
:root {{
  --a1: {a1};
  --a2: {a2};
}}
</style>
"""
st.markdown(accent_css, unsafe_allow_html=True)
st.markdown(LIGHT if st.session_state["light_mode"] else DARK, unsafe_allow_html=True)
```

```python
st.markdown('<div class="hero">Poem Studio Ultra — NLP Poem Generator</div>', unsafe_allow_h
st.caption("Give *any* word/phrase → pick a mood → get a meaningful poem (rhyme + imagery)."

# Tabs: Compose | History | About
compose_tab, history_tab, about_tab = st.tabs(["🎤 Compose", "🕘 History", "ℹ️ About"])

# ---------- Compose Tab ----------
with compose_tab:
    left, right = st.columns([1.1, 1])

    with left:
        st.markdown('<div class="card">', unsafe_allow_html=True)
        st.subheader("⚙️ Controls")

        # Quick preset chips (optional shortcut)
        chip_cols = st.columns(6)
        for i, p in enumerate(["love", "friendship", "nature", "memory", "hope", "time"]):
            if chip_cols[i].button(p.capitalize(), key=f"chip_{p}"):
                st.session_state["theme_word"] = p

        theme_word = st.text_input(
            "Theme (any word/phrase)",
```

```python
        emotion = st.selectbox("Emotion / tone", ["calm", "happy", "romantic", "inspiring",
        lines = st.selectbox("Lines", [6, 8, 12], index=0)
        rhyme = st.selectbox("Rhyme scheme", ["AABB", "ABAB"], index=0)
        st.markdown('</div>', unsafe_allow_html=True)

        st.markdown('<div class="card">', unsafe_allow_html=True)
        c1, c2, c3 = st.columns([1, 1, 1])
        with c1:
            go = st.container().button("Generate Poem", type="primary")
        with c2:
            pdf_slot = st.empty()
        with c3:
            img_slot = st.empty()
        st.markdown('</div>', unsafe_allow_html=True)

    with right:
        st.markdown('<div class="card">', unsafe_allow_html=True)
        poem_box = st.empty()
        st.markdown('</div>', unsafe_allow_html=True)

# ---- Helpers: grammar & meter
def polish(text: str) -> str:
    if not text:
```

```python
        return text
    text = re.sub(r"\s+([,.;:!?])", r"\1", text)
    text = re.sub(r"([,.;:!?])([A-Za-z])", r"\1 \2", text)
    ls = [ln.rstrip() for ln in text.splitlines()]
    out = []
    for i, ln in enumerate(ls):
        if i == 0:
            out.append(ln.strip()); continue
        if ln and ln[-1] not in ".?!…":
            ln += "."
        if ln and ln[0].islower():
            ln = ln[0].upper() + ln[1:]
        out.append(ln)
    return "\n".join(out)

def syllables_in_word(w: str) -> int:
    w = re.sub(r"[^a-zA-Z]", "", (w or "").lower())
    if not w: return 0
    if _CMU and w in _CMU:
        phones = _CMU[w][0]
        return sum(ch[-1].isdigit() for ch in phones)
    groups = re.findall(r"[aeiouy]+", w)
    return max(1, len(groups))
```


```python
    groups = re.findall(r"[aeiouy]+", w)
    return max(1, len(groups))
```

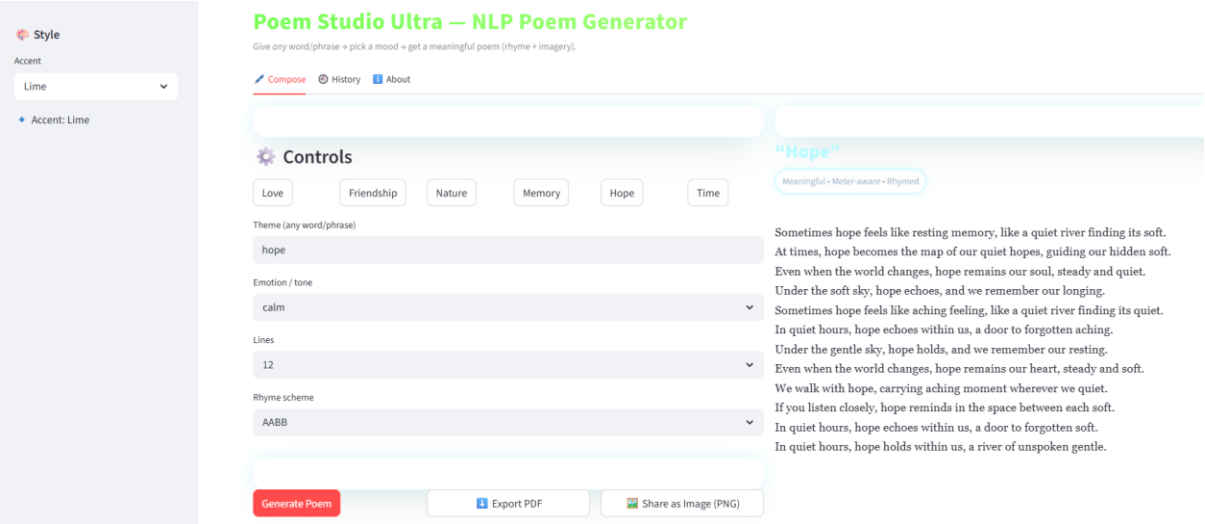

```python

def syllables_in_line(ln: str) -> int:
    return sum(syllables_in_word(x) for x in re.findall(r"[A-Za-z']+", ln))

def generate_line(theme: str, emotion: str, rhyme: str) -> str:
    txt = generate_poem(theme, lines=1, emotion=emotion, rhyme_scheme=rhyme)
    parts = txt.splitlines()
    return parts[1] if len(parts) > 1 else (parts[0] if parts else "")

def enforce_meter(lines_list, theme, emotion, rhyme, target=10, tol=2, max_tries=12):
    shaped = []
    for ln in lines_list:
        tries, curr = 0, ln
        while tries < max_tries:
            s = syllables_in_line(curr)
            if target - tol <= s <= target + tol: break
            curr = generate_line(theme, emotion, rhyme)
            tries += 1
        shaped.append(curr)
    return shaped
```

```python
poem_title, poem_body = None, None
if "history" not in st.session_state: st.session_state["history"] = []

if go:
    if not (theme_word or "").strip():
        st.warning("Please enter any theme/word/phrase in the left panel.")
    else:
        raw = generate_poem(theme_word, lines=int(lines), emotion=emotion, rhyme_scheme=r
        pretty = polish(raw)
        parts = pretty.splitlines()
        poem_title = parts[0].strip() if parts else "Poem"
        body_lines = parts[1:] if len(parts) > 1 else []
        shaped_lines = enforce_meter(body_lines, theme_word, emotion, rhyme, target=10, t
        poem_body = "\n".join(shaped_lines)

        st.session_state["history"].insert(0, {"title": poem_title, "body": poem_body})
        st.session_state["history"] = st.session_state["history"][:12]

if poem_title and poem_body:
    with poem_box.container():
        st.markdown('<div class="title">' + f'“{poem_title}”' + '</div>', unsafe_allow_ht
        st.markdown('<span class="badge">Meaningful • Meter-aware • Rhymed</span>', unsaf
        st.markdown('<br/>', unsafe_allow_html=True)
```

```python
def poem_to_pdf_bytes(title: str, body: str) -> io.BytesIO:
    from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.units import inch
    from reportlab.lib import colors
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont

    fonts_dir = Path("assets/fonts")
    title_font = None
    if (fonts_dir / "GreatVibes-Regular.ttf").exists():
        pdfmetrics.registerFont(TTFont("GreatVibes", str(fonts_dir / "GreatVibes-Regu
        title_font = "GreatVibes"
    body_font = None
    if (fonts_dir / "PlayfairDisplay-Regular.ttf").exists():
        pdfmetrics.registerFont(TTFont("Playfair", str(fonts_dir / "PlayfairDisplay-R
        body_font = "Playfair"

    buff = io.BytesIO()
    doc = SimpleDocTemplate(buff, pagesize=A4, leftMargin=60, rightMargin=60, topMarg
    styles = getSampleStyleSheet()
    title_style = ParagraphStyle("PoemTitle", parent=styles["Title"],
```

```python
with history_tab:
    st.markdown('<div class="card">', unsafe_allow_html=True)
    st.subheader("Recent poems")
    if st.session_state.get("history"):
        for i, item in enumerate(st.session_state["history"]):
            with st.expander(f"{i+1}. {item['title']}"):
                st.text(item["body"])
    else:
        st.info("No history yet. Generate your first poem in the Compose tab.")
    st.markdown('</div>', unsafe_allow_html=True)

# ---------- About Tab ----------
with about_tab:
    st.markdown('<div class="card">', unsafe_allow_html=True)
    st.subheader("About this app")
    st.markdown("""
- **NLP engine**: WordNet theme typing → curated vocabulary → poetic templates → rhyme en
- **Meter shaping**: optional CMUdict syllable counts (fallback heuristic) to approximate
- **Export**: typeset PDF and a poster-style PNG with rounded card + ribbons.
- **Theme**: Live Dark/Light toggle with accent color selection.
    """)
    st.markdown('</div>', unsafe_allow_html=True)
```

OUTPUT



CONCLUSION

The development of a *Poem Generator Based on Theme using Natural Language Processing* represents a significant step forward in the field of computational creativity. By integrating advanced NLP techniques such as neural networks, contextual embeddings, and transformer-based architectures, the system can produce meaningful, coherent, and thematically relevant poetry. This project demonstrates that machines can learn the complexities of human language, not only in terms of grammar and vocabulary but also in artistic expression, emotional tone, and literary structure.

Through this work, we observe how AI models can analyse a given theme and translate it into poetic content that reflects the intended mood and style. Although the generated poems may still lack the depth, cultural nuance, and emotional subtlety of human-created literature, the system successfully showcases the potential of AI as an assistive creative tool. It highlights the evolving relationship between technology and art, illustrating that artificial intelligence can enhance human creativity rather than replace it. Overall, this project serves as a foundation for more advanced systems that combine linguistic intelligence with artistic innovation.